

Variable, Literal, Data Type, and Constant

Variable, Literal, Data Type, Constant in details ?

Absolutely! Let's expand on each concept in detail with examples, rules, and use cases — this will give you a solid understanding of how these key parts of Java work together.

✔ 1. Variable

◆ Definition:

A **variable** is a **name given to a memory location where data is stored**. It allows us to store, modify, and retrieve values during the execution of a program.

◆ Characteristics:

- Every variable has a **data type, name, and value**.
- The value of a variable **can change** during program execution (unless it's final).
- Variables must be **declared before use**.

◆ Syntax:

```
dataType variableName = value;
```

◆ Example:

```
int age = 25;  
String name = "Java";  
boolean isRunning = true;
```

◆ Types of Variables:

Type	Scope	Memory	Declared inside
Local variable	Inside a method/block	Stack	Method
Instance variable	For each object (non-static)	Heap	Class
Static variable	Shared among all objects	Method Area	Class (with static)

✔ 2. Literal

◆ Definition:

A **literal** is a **fixed value written directly in the code**. It represents a constant value assigned to a variable.

◆ Types of Literals:

Type	Example	Explanation
Integer	10, 0xA, 07	Decimal, Hexadecimal, Octal

Type	Example	Explanation
Float	3.14f, 5.0F	Must end with f or F
Double	2.718, 1.0d	Optional d/D, default for decimals
Char	'A', '1', '\n'	Must be in single quotes
Boolean	true, false	Logical true/false
String	"Hello"	Sequence of characters in double quotes
Null	null	Represents a null reference

◆ **Example:**

```
int a = 10;    // 10 is an integer literal
double pi = 3.14; // 3.14 is a double literal
char ch = 'A'; // 'A' is a character literal
String name = "Bob"; // "Bob" is a string literal
```

✓ 3. Data Type

◆ **Definition:**

A **data type** defines the **type of data** a variable can hold, such as integers, decimals, characters, or boolean values.

◆ **Categories:**

1. **Primitive Data Types** (8 types: int, float, char, etc.)
2. **Non-Primitive Data Types** (e.g., String, Array, Class)

◆ **Example:**

```
int age = 25;    // integer type
double salary = 5000.50; // floating-point type
char grade = 'A'; // character type
boolean result = true; // boolean type
String name = "Alice"; // object (non-primitive) type
```

✓ 4. Constant

◆ **Definition:**

A **constant** in Java is a variable whose **value cannot be changed** once assigned. You use the **final** keyword to declare a constant.

◆ **Syntax:**

```
final dataType VARIABLE_NAME = value;
```

💡 By convention, constants are written in **UPPERCASE**.

◆ **Example:**

```
final double PI = 3.14159;
final int MAX_USERS = 100;
```

If you try to change the value of a final variable, the compiler will throw an error:

```
PI = 3.14; // ✗ Error: cannot assign a value to final variable
```

★ Real Example (All Concepts Together):

```
public class ConceptsDemo {  
  
    // Constant  
    static final double PI = 3.14159;  
  
    // Instance variable  
    String language = "Java";  
  
    public static void main(String[] args) {  
        // Local variables with literals  
        int age = 22;           // int variable with literal  
        char grade = 'A';      // char literal  
        boolean isPassed = true; // boolean literal  
        String name = "Alice";  // string literal  
  
        // Printing variables  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Grade: " + grade);  
        System.out.println("Passed: " + isPassed);  
        System.out.println("Value of PI: " + PI);  
    }  
}
```

🔍 Quick Comparison Table:

Concept	Meaning	Can Change?	Keyword used	Example
Variable	Container for data	✔ Yes	-	int age = 25;
Literal	Actual value assigned	✗ No	-	"Java", 3.14, 'A'
Data Type	Defines kind of data stored	✔ Yes	-	int, double, String
Constant	Fixed variable (unchangeable)	✗ No	final	final int MAX = 100;

What is a Variable in Java?

✔ What is a Variable in Java?

A **variable** in Java is a **named container** (or memory location) used to **store data** that can be used and changed during the execution of a program.

◆ Think of a Variable As:

🧠 A label for a **memory box** where values are stored.

◆ Syntax of Declaring a Variable

```
dataType variableName = value;
```

◆ Example:

```
int age = 25;           // age is a variable of type int
String name = "Alice"; // name is a variable of type String
```

◆ Types of Variables in Java

Type	Where it's declared	Scope	Default Value
Local Variable	Inside a method or block	Only within that method/block	No default — must initialize
Instance Variable	Inside class, but outside methods	Available for each object	Depends on data type
Static Variable	Inside class with static keyword	Shared among all objects	Depends on data type

◆ Example Code

```
public class VariableDemo {

    // Instance variable
    String language = "Java";

    // Static variable (shared)
    static int version = 21;

    public static void main(String[] args) {
        // Local variable
        int age = 25;

        // Print all variables
        System.out.println("Local variable: " + age);
        System.out.println("Static variable: " + version);
    }
}
```

```

    // Create object to access instance variable
    VariableDemo obj = new VariableDemo();
    System.out.println("Instance variable: " + obj.language);
}
}

```

◆ Rules for Naming Variables

- Must start with a letter, `_`, or `$`
 - Cannot start with a digit
 - Cannot use keywords (like `int`, `class`, etc.)
 - Use **camelCase** by convention (e.g., `studentName`)
-

◆ Why Use Variables?

- To **store and reuse data**
 - To **perform calculations**
 - To **manipulate user input**
 - To make your code **flexible and readable**
-

Three separate complete Java programs, each one demonstrating a different type of variable:

Absolutely! Below are **three separate complete Java programs**, each one demonstrating a **different type of variable**:

✓ 1. Local Variable

Here is the **detailed theory on local variables** in Java, suitable for learning documentation or interview preparation:

📖 Local Variables in Java – Theory

◆ What is a Local Variable?

A **local variable** is a variable that is declared **inside a method, constructor, or block**, and is **only accessible within that scope**. It **lives temporarily** in memory while the method/block is running and is destroyed once it ends.

◆ Key Characteristics of Local Variables:

Property	Description
Belongs to	The method or block in which it's declared
Accessed by	Only within the method/block
Stored in	Stack memory

Property	Description
Lifetime	Created when the method/block starts, destroyed when it ends
Default value	✗ No default value – must be initialized before use
Scope	Limited to the enclosing method or block
Sharing across methods	Not shared – accessible only in declared method

◆ Behavior of Local Variables:

- Cannot be accessed outside the method or block where they are declared.
 - Must be **explicitly initialized** before use; otherwise, the code will not compile.
 - Unlike static and instance variables, local variables do **not have default values**.
 - Re-declaring a variable with the same name in nested blocks **shadows** the outer variable.
-

◆ Syntax Example:

```
public class LocalVariableExample {

    public void display() {
        int age = 25; // local variable
        System.out.println("Age is: " + age);
    }

    public static void main(String[] args) {
        LocalVariableExample obj = new LocalVariableExample();
        obj.display();
    }
}
```

◆ Real-World Analogy:

Think of a **local variable** like a **temporary note** you write during a meeting. Once the meeting is over (method ends), the note (variable) is thrown away — it doesn't exist anymore.

◆ Must Be Initialized Before Use:

```
public void check() {
    int count;        // Declared
    // System.out.println(count); ✗ Error: Variable 'count' might not have been initialized
    count = 10;       // Initialized
    System.out.println(count); // ✓ Now it's okay
}
```

◆ Scope Limitation Example:

```
public void testScope() {
    int number = 10;
    if (number > 5) {
        int temp = 100; // local to 'if' block
        System.out.println(temp);
    }
    // System.out.println(temp); ✗ Error: temp cannot be resolved
}
```

}

◆ Use Case:

Use **local variables** when:

- You need a temporary variable **just for the method's logic**.
- You want to avoid polluting the object's state or global memory.

✔ Summary Table: Local vs Instance vs Static Variables

Feature	Local Variable	Instance Variable	Static Variable
Belongs To	Method/block	Object	Class
Memory Location	Stack	Heap	Method Area
Lifetime	During method/block only	As long as object lives	As long as class is loaded
Default Initialization	✗ Must initialize manually	✔ Auto-initialized	✔ Auto-initialized
Access Scope	Inside method/block only	Anywhere in class (object)	Anywhere in class

🔍 Important Points for Interviews:

- Local variables **must be initialized** before use.
- They are **not accessible** outside the method/block.
- Memory-efficient due to their temporary nature (allocated on the **stack**).
- Great for **function-specific temporary calculations** or counters.

🧠 **Note:** Local variables do **not get default values** — you must initialize them before use.

✔ 2. Instance Variable

An **instance variable** in Java is a **non-static** variable declared inside a class but **outside of any method or constructor**. Each time you create a new object of the class, a **new and separate copy** of that instance variable is created for that object.

◆ Key Characteristics of Instance Variables:

Property	Description
Belongs to	An object (instance) of the class
Accessed by	Using the object reference
Stored in	Heap memory (as part of the object)
Lifetime	Lives as long as the object exists
Default values	Automatically initialized (e.g., null for objects, 0 for numbers)

Property	Description
Scope	Accessible throughout the class (non-static context)
Sharing across objects	Not shared – each object has its own copy

◆ Behavior of Instance Variables:

- When you create **multiple objects**, **each object gets its own independent copy** of the instance variable.
 - If you change the value of an instance variable using one object, **it does not affect** the instance variable in another object.
 - This is in contrast to **static variables**, which are shared among all objects of the class.
-

◆ Why Are They Called "Instance" Variables?

They are called *instance* variables because they are tied to a **specific instance** of the class. Unlike class-level variables (static), instance variables hold **individual state** for each object.

◆ Real-World Analogy:

Think of a class as a blueprint for a car, and color as an instance variable. Every car (object) created from that blueprint can have a different color.

```
Car1.color = "Red"  
Car2.color = "Blue"
```

Changing Car2.color to "Green" will not affect Car1.color.

◆ Use Case:

Use instance variables when:

- Each object needs to store **its own data**.
 - The variable represents a **property or attribute** of the object.
-

◆ Example Code:

```
public class InstanceVariableExample {  
  
    // Instance variable  
    String name = "Alice";  
}
```



```

public void showName() {
    System.out.println("Instance variable value: " + name);
}

public static void main(String[] args) {
    // Create two objects
    InstanceVariableExample obj1 = new InstanceVariableExample();
    InstanceVariableExample obj2 = new InstanceVariableExample();

    // Change name for obj2
    obj2.name = "Bob";

    // Each object has its own value
    obj1.showName(); // Output: Alice
    obj2.showName(); // Output: Bob
    obj1.showName(); // Output: Alice
}
}

```

 **Note:** Instance variables **can have default values** if not initialized.

✓ 3. Static Variable

Here is the **detailed theory about static variables** in Java, ideal for documentation or interview preparation:

■ Static Variables in Java – Theory

◆ What is a Static Variable?

A **static variable** is a **class-level variable** declared with the static keyword inside a class but **outside any method, constructor, or block**. It is **shared among all instances (objects)** of the class and is loaded into memory **only once** when the class is loaded.

◆ Key Characteristics of Static Variables:

Property	Description
Belongs to	The class itself (not to any object)
Accessed by	Using the class name or any object reference (preferably class name)
Stored in	Method Area (a part of JVM memory)
Lifetime	As long as the class is loaded in memory
Default values	Automatically initialized (like instance variables)
Scope	Accessible throughout the class (in both static and non-static contexts)
Sharing across objects	Shared – Only one copy exists for all objects of the class

Behavior of Static Variables:

- A static variable is **common to all objects** of the class.
- If one object modifies the static variable, **all other objects see the updated value**.

- You can access it even **without creating an object**, using the **class name**.
-

◆ Why Use Static Variables?

- To store **common/shared data** across all instances.
 - To reduce memory usage, since only **one copy** exists in memory.
 - Useful for constants, counters, configuration values, etc.
-

◆ Syntax Example:

```
public class Employee {
    static String company = "TechCorp"; // Static variable
    String name; // Instance variable

    // Constructor
    Employee(String name) {
        this.name = name;
    }

    // Method to display details
    public void showDetails() {
        System.out.println("Name: " + name);
        System.out.println("Company: " + company);
    }

    // Main method
    public static void main(String[] args) {
        // Create two employees
        Employee emp1 = new Employee("Alice");
        Employee emp2 = new Employee("Bob");

        // Display details
        emp1.showDetails();
        emp2.showDetails();

        // Change static variable
        Employee.company = "NewTech";

        // Display again to see effect of static change
        emp1.showDetails();
        emp2.showDetails();
    } }
```

```
Name: Alice
Company: TechCorp
Name: Bob
Company: TechCorp
Name: Alice
Company: NewTech
Name: Bob
Company: NewTech
```

◆ Real-World Analogy:

Imagine a **school** where:

- Each **student** has a unique name → Instance variable.
- All students belong to the same schoolName → Static variable.

Student1.schoolName = "ABC School"
 Student2.schoolName = "ABC School"

If Student1 changes schoolName to "XYZ School", then Student2 sees "XYZ School" too.

◆ Example Use Case:

```
public class Counter {
    static int count = 0; // Static variable

    Counter() {
        count++;
        System.out.println("Object created, count = " + count);
    }

    public static void main(String[] args) {
        Counter c1 = new Counter(); // count = 1
        Counter c2 = new Counter(); // count = 2
        Counter c3 = new Counter(); // count = 3
    }
}
```

Output:

Object created, count = 1
 Object created, count = 2
 Object created, count = 3

This demonstrates that count is shared among all objects.

✔ Summary Table: Static vs Instance Variable

Feature	Static Variable	Instance Variable
Belongs To	Class	Each object
Accessed By	Class name or object	Object only
Memory Location	Method Area	Heap
Number of Copies	One shared copy	Separate copy per object
Updated By	One update reflects in all objects	Update affects only that object
Use Case	Shared settings, constants, counters	Object-specific data (like name)

 **Note:** Static variables are stored in the **Method Area** and have only **one copy per class**, not per object.

🔄 Summary Table:

Type	Defined Where	Access Level	Default Value	Keyword
Local Variable	Inside method or block	Only within that block	✗ None	—
Instance Variable	In class, outside methods	Per object (needs object)	✓ Yes	—
Static Variable	In class with static keyword	Shared among all objects	✓ Yes	static

Would you like to see all three types combined in one single program as well?

Awesome! Here's a **single complete Java program** that shows how **local**, **instance**, and **static** variables all work together in one place — with comments to help you understand the flow.

✓ Java Program: All Variable Types Together

```
public class AllVariablesExample {

    // Static variable (shared by all objects)
    static int staticCounter = 0;

    // Instance variable (unique to each object)
    String instanceName;

    // Constructor to initialize instance variable and increment static counter
    public AllVariablesExample(String name) {
        instanceName = name;
        staticCounter++; // shared variable gets incremented every time an object is created
    }

    public void showDetails() {
        // Local variable (only exists inside this method)
        int localScore = 100;

        System.out.println("Instance variable (Name): " + instanceName);
        System.out.println("Static variable (Counter): " + staticCounter);
        System.out.println("Local variable (Score): " + localScore);
        System.out.println("-----");
    }

    public static void main(String[] args) {
        // Create 3 objects
        AllVariablesExample obj1 = new AllVariablesExample("Alice");
        AllVariablesExample obj2 = new AllVariablesExample("Bob");
        AllVariablesExample obj3 = new AllVariablesExample("Charlie");

        // Call method to display details
        obj1.showDetails();
        obj2.showDetails();
        obj3.showDetails();

        // Static variable accessed using class name (preferred way)
        System.out.println("Final static count (shared): " + AllVariablesExample.staticCounter);
    }
}
```

✓ Output:

Instance variable (Name): Alice
Static variable (Counter): 3
Local variable (Score): 100

Instance variable (Name): Bob
Static variable (Counter): 3
Local variable (Score): 100

Instance variable (Name): Charlie
Static variable (Counter): 3
Local variable (Score): 100

Final static count (shared): 3

Recap of What's Happening:

Variable Type	Where It's Used	Behavior
Local	Inside method showDetails	Re-created every time method runs
Instance	In each object	Separate value per object
Static	Shared across all objects	Same value for all objects

You got it! ¹⁰⁰ Here's the **complete list of rules** for writing **Java variable names**, including rules about **keywords**, **syntax**, **conventions**, and **special cases** — everything you need to master it like a pro.

✓ All Rules for Writing Variable Names in Java

1. Syntax Rules (Compiler-Enforced — Must Follow)

Rule No.	Rule Description	✓ Example	✗ Invalid Example
1	Variable name must start with a letter , _ (underscore), or \$ (dollar sign)	name, _age, \$x	1name, @rate
2	Cannot start with a digit	—	2value
3	Can include letters , digits , _ , and \$	data23, _temp	total#count
4	Cannot contain spaces	—	user name
5	Cannot use Java reserved keywords as names	—	int, class, new
6	Java is case-sensitive — Age, age, and AGE are different	total, Total	—
7	Cannot use operators or special characters	—	value@num, sum%tax

Rule No.	Rule Description	✓ Example	✗ Invalid Example
8	No limit on variable length (but avoid too long)	totalAmountWithTax —	
9	var is reserved in Java 10+ for type inference, avoid using it as a name	var x = 5;	int var = 5; (allowed but discouraged)

🚫 2. Java Reserved Keywords (Cannot Use as Variables)

These are reserved for Java syntax:

abstract	assert	boolean	break	byte
case	catch	char	class	const
continue	default	do	double	else
enum	extends	final	finally	float
for	goto	if	import	implements
instanceof	int	interface	long	native
new	package	private	protected	public
return	short	static	strictfp	super
switch	synchronized	this	throw	throws
transient	try	void	volatile	while
record	sealed	permits	non-sealed	yield
var				

! const, goto, and var are reserved and cannot be used, even if they aren't currently active in Java.

Absolutely! Below is a **complete categorization and explanation** of Java's **reserved keywords** grouped by their **functionality or working area**, which is ideal for documentation and interview prep.

📁 Reserved Keywords in Java (Grouped by Functionality)

◆ 1. Access Modifiers (Control access level of classes, methods, variables)

Keyword	Description
public	Accessible from anywhere
private	Accessible within the same class only
protected	Accessible in the same package or subclasses
default	(not a keyword for access) Package-private (no modifier)

◆ 2. Data Types (Primitive data types)

Keyword	Description
byte	8-bit integer
short	16-bit integer
int	32-bit integer
long	64-bit integer
float	32-bit floating point
double	64-bit floating point
char	Single 16-bit Unicode char
boolean	true or false values

◆ 3. Flow Control (Conditional/Looping)

Keyword	Description
if	Executes a block if condition is true
else	Executes if the if condition is false
switch	Selects between multiple cases
case	A branch in switch
default	Default branch in switch
for	Looping (with init, condition, update)
while	Looping as long as condition is true
do	Looping: executes at least once
break	Exit loop or switch
continue	Skip to next loop iteration
return	Exit method and return value
yield	Used in switch expressions (Java 13+)

◆ 4. Exception Handling

Keyword	Description
try	Start block of code that may throw error
catch	Handle exception
finally	Always executes after try/catch
throw	Throw an exception
throws	Declares method may throw exceptions

◆ 5. Modifiers (Class/Method/Variable Behavior)

Keyword	Description
static	Belongs to class, not object
final	Constant or can't be overridden
abstract	Method/class without full implementation
synchronized	Only one thread at a time
volatile	Prevents thread caching of variable
transient	Not serializable
native	Implemented in another language (like C)
strictfp	Floating-point consistency across platforms

◆ 6. Object-Oriented Concepts

Keyword	Description
class	Defines a class
interface	Contract for classes
extends	Inherit from a class
implements	Implement an interface
new	Create object
this	Reference current object
super	Reference parent class
instanceof	Check if object is of certain type

◆ 7. Package/Import Management

Keyword	Description
package	Defines package for class
import	Brings other classes/packages in

◆ 8. Uncategorized/Special Purpose

Keyword	Description
assert	Assertion for debugging
enum	Defines a fixed set of constants (enumeration)
record	Compact class for immutable data (Java 14+)
sealed	Limits which classes can extend (Java 15+)
permits	Works with sealed to declare allowed subclasses
non-sealed	Breaks sealing chain, allows open inheritance
var	Type inference (Java 10+)

◆ 9. Reserved but Unused

Keyword	Description
goto	Reserved but not used
const	Reserved (use final instead)

✔ Summary Table

Category	Examples
Access Modifiers	public, private, protected
Data Types	int, double, char, boolean, etc.
Control Statements	if, for, switch, break, return, etc.
Exception Handling	try, catch, finally, throw, throws
OOP Keywords	class, extends, this, super, etc.
Modifiers	static, final, abstract, synchronized
Packages and Imports	package, import
Special Features	enum, record, sealed, var, etc.

Category	Examples
Unused Reserved	goto, const

✳ 3. Best Practices / Naming Conventions (Recommended)

These make your code more readable and maintainable:

Convention	Description	Example
🐫 camelCase	Use for variable names: start with lowercase, capitalize next words	userName, taxRate
🔊 UPPER_CASE	Use for final constants	MAX_SPEED, PI
★ Meaningful names	Avoid meaningless names like a, x1	✓ totalAmount
📦 Nouns for vars	Use nouns, not verbs, for variables	✓ age, ✗ calculate
🔪 Short but clear	Keep it short and descriptive	✓ score, ✗ thisIsTotalSumOfAllMarksEver
🔧 Use prefixes for context	Helps avoid keyword clashes	className, intValue

🔍 4. Special Notes

Case	Allowed?	Example
Variable name = class name	✓ (but confusing)	int String = 5; (allowed but not recommended)
Variable name = method name	✓	int print = 10;
Using var	✓ Java 10+	var x = 5; (type inferred)
Unicode characters	✓	int कक्षा = 10; (allowed but not advised)
Variable name same as object	✓	Scanner Scanner = new Scanner(System.in);

📄 Quick Summary Table

✓ Do This	✗ Don't Do This
Use camelCase	Use PascalCase for variables

✔ Do This

Use meaningful names

Use letters, digits, _, \$

Avoid reserved keywords

Use constants in UPPER_CASE

✗ Don't Do This

Use single-letter names like x

Use @, %, #, *, &

Name variable int, class, etc.

Name constants like piValue

New

Sure! Here's a **detailed explanation of Java variable naming rules**, along with examples, reasoning, and tips to make it easier to understand:

✔ 1. Case Sensitive

- Java is **case sensitive**, meaning amount, Amount, and AMOUNT are all **different** identifiers.

💡 Example:

```
int amount = 100;
int Amount = 200;
System.out.println(amount); // 100
System.out.println(Amount); // 200
```

⚠ Be careful! A small change in letter case can create bugs or unexpected behavior.

✔ 2. Allowed Characters in Variable Names

A variable name **can contain**:

- Alphabets (A-Z, a-z)
- Numbers (0-9)
- Underscore _
- Dollar sign \$ (rarely used)

💡 Example:

```
int room1 = 101;
int cabin51 = 302;
int room_number = 105;
float total$amount = 599.99f;
```

⚠ Even though \$ is allowed, it's **not recommended** for regular variables (used by compilers and generated code sometimes).

✔ 3. Must Start With:

A variable **must start with**:

- A letter (A-Z or a-z)
- An underscore _
- A dollar sign \$

✗ *Cannot start with a number:*

int 1number = 10; // ✗ Invalid

int number1 = 10; // ✓ Valid

◆ *Valid Examples:*

byte x1 = 5;

byte _x = 10;

byte \$x = 15;

✓ 4. Cannot Use Java Keywords

Java has **reserved words** (called **keywords**) that cannot be used as variable names.

◆ *Examples of keywords:*

int int = 5; // ✗ Invalid

float class = 4.5f; // ✗ Invalid

✓ *Valid:*

int total = 5;

◆ **Common keywords you can't use:** int, class, public, static, void, if, else, new, etc.

✓ 5. Avoid Naming Conflict with Class Names (if in use)

You **can** use class names as variable names **unless** the class is being actively used in the same scope — **then it may cause confusion or error.**

✗ *Confusing:*

String String = "Hello"; // Works but confusing, avoid it

It's a **bad practice** because it makes the code harder to understand.

✓ 6. No Limit on Length

You can name your variable with **any length** — long or short.

◆ *Examples:*

int a = 10; // ✓ Valid

int numberOfStudentsInFirstYear = 200; // ✓ Valid

✧ Keep variable names **meaningful and readable**, even if long.

✓ 7. Camel Case Naming Convention

Camel case makes names **more readable**:

- First word → **starts lowercase**
- Following words → **start with uppercase**

🔗 *Examples:*

```
int rollNumber;
String studentName;
float totalAmountPaid;
```

⊘ Don't do this:

```
int Rollnumber; // ✗
int rollnumber; // ✓ but less readable than rollNumber
```

🌟 **Best Practices for Naming Variables:**

Practice	Example
Meaningful names	studentAge, totalAmount
Follow camel case	userEmail, itemPrice
No abbreviations	totalAmount not totAmt
Avoid single-letter names	Use counter not c (unless in loops)
Avoid confusing names	Don't name variable String, System, etc.

✓ **Summary:**

Rule	Description
Case-sensitive	Amount ≠ amount
Allowed characters	A-Z, a-z, 0-9, _, \$
Start with	Letter, _, or \$
No keywords	int int = 5; is invalid
Avoid class name conflicts	Especially String, List, etc.
No length limit	But keep it meaningful
Camel casing	rollNumber, studentName

What is a Literal in Java?

✓ **What is a Literal in Java?**

A **literal** in Java is a **fixed value** directly written in the code that is **assigned to a variable**. It's the actual **constant value** you use in your program.

◆ Simple Definition:

📖 A **literal** is the real value you put inside a variable.

For example:

```
int age = 25;
```

- age → variable
 - 25 → **literal**
-

◆ Types of Literals in Java

Java supports several **types of literals** depending on the data type:

Literal Type	Example	Description
Integer	10, 0x1A, 075	Decimal, Hex, Octal
Floating Point	3.14f, 6.28	Float & double values
Character	'A', '1', '\n'	Single character inside ''
String	"Hello"	Sequence of characters inside ""
Boolean	true, false	Logical values
Null	null	Special literal for object references

◆ Examples:

```
int number = 100;      // 100 is an integer literal
float rate = 5.75f;    // 5.75f is a float literal
char grade = 'A';      // 'A' is a character literal
boolean passed = true; // true is a boolean literal
String message = "Hello"; // "Hello" is a string literal
```

◆ Notes:

- Integer literals **default to int**
 - Decimal literals **default to double**
 - For float, you **must use f or F** (e.g., 2.5f)
 - For long, use L or l (e.g., 100000L)
 - char literals must be in **single quotes** (e.g., 'B')
-

◆ Bonus: Special Literals

Literal	Description
null	Represents no object

Literal	Description
'\n', '\t'	Escape sequences in char/String

✓ **Real Example (with multiple literals):**

```
public class LiteralExample {  
    public static void main(String[] args) {  
        int age = 30;           // integer literal  
        float price = 99.99f;   // float literal  
        char letter = 'J';      // char literal  
        boolean isValid = true; // boolean literal  
        String greeting = "Welcome"; // string literal  
  
        System.out.println(age);  
        System.out.println(price);  
        System.out.println(letter);  
        System.out.println(isValid);  
        System.out.println(greeting);  
    }  
}
```

What is a Constant in Java?

Great question! Let's break it down clearly:

✔ What is a Constant in Java?

A **constant** in Java is a variable whose **value cannot be changed** once it is assigned.

🔒 It's like a *read-only* variable — set it once, and it stays the same forever.

◆ How to Create a Constant in Java?

You use the **`final`** keyword to define a constant.

◆ Syntax:

```
final dataType variableName = value;
```

◆ Example:

```
final int SPEED_LIMIT = 60;
```

- `final` → makes it a constant
 - `SPEED_LIMIT` → variable name (by convention, **constants are written in UPPERCASE**)
 - `60` → fixed value that can't be changed later
-

◆ Attempting to Change a Constant

```
public class ConstantExample {  
    public static void main(String[] args) {  
        final int MAX_USERS = 100;  
        System.out.println("Max Users: " + MAX_USERS);  
  
        // MAX_USERS = 200; // ✗ Compilation error: cannot assign a value to final variable  
    }  
}
```

- Trying to assign a new value to `MAX_USERS` will cause a **compile-time error**.
-

◆ Where Constants Are Commonly Used

- For **fixed configuration values**
 - In **mathematical formulas**
 - To improve **code readability and maintainability**
-

◆ Real-world Constant Example:

```
final double PI = 3.14159;  
double radius = 5;
```



```
double area = PI * radius * radius;
System.out.println("Area of Circle: " + area);
```

◆ Summary Table

Feature	Description
Keyword	final
Mutability	✗ Cannot be changed once assigned
Naming	UPPERCASE by convention
Use case	Prevent accidental modification of important values

◆ Tip: static final = Constant shared by all objects

```
class AppConfig {
    static final String APP_NAME = "MyApp";
}
```

You can access it without creating an object:

```
System.out.println(AppConfig.APP_NAME);
```

Would you like an example showing constants in a real-world mini project (like tax calculator, school system, etc.)?

Awesome! Let's create a **real-world mini project** using **constants** in Java — we'll go with a **Simple Tax Calculator** 🚀

✓ Mini Project: Tax Calculator Using Constants

We'll define:

- A **constant tax rate**
 - Calculate total tax and net salary
 - Use final and static final for constants
-

◆ Code Example:

```
public class TaxCalculator {

    // Constant tax rate (shared across all objects)
    static final double TAX_RATE = 0.15; // 15%

    public static void main(String[] args) {

        // Input salary (can be taken from user using Scanner as well)
        double grossSalary = 50000;

        // Calculate tax and net salary
        double taxAmount = grossSalary * TAX_RATE;
        double netSalary = grossSalary - taxAmount;
```

```

    // Print results
    System.out.println("Gross Salary: $" + grossSalary);
    System.out.println("Tax Rate: " + (TAX_RATE * 100) + "%");
    System.out.println("Tax Amount: $" + taxAmount);
    System.out.println("Net Salary: $" + netSalary);
}
}

```

✓ Output:

```

Gross Salary: $50000.0
Tax Rate: 15.0%
Tax Amount: $7500.0
Net Salary: $42500.0

```

🧠 Why Use Constants Here?

Item	Value	Why final?
TAX_RATE	0.15	Fixed percentage for tax
static	Yes	Shared value, no need to create object

If tax rate changes, update only **one line** → cleaner and safer code!

🔗 Optional: Add Scanner for Dynamic Salary Input

Would you like to see this example extended to accept **user input using Scanner** or have it split into **methods and classes** for real structure?

Great! Let's enhance our **Tax Calculator mini project** with the following:

✓ Features we'll add:

- 🗣️ User input using Scanner
 - ⚙️ Constants defined with static final
 - 📁 Organized into methods
 - 😊 Cleaner output formatting
-

✓ Final Tax Calculator (with Scanner + Constants + Methods)

```

import java.util.Scanner;

public class TaxCalculator {

    // Constant tax rate (15%)
    static final double TAX_RATE = 0.15;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // Scanner object for user input

        System.out.print("Enter your gross salary: $");
        double grossSalary = scanner.nextDouble();
    }
}

```

```

double tax = calculateTax(grossSalary);
double netSalary = grossSalary - tax;

displayResults(grossSalary, tax, netSalary);

scanner.close(); // Always close the scanner
}

// Method to calculate tax
public static double calculateTax(double salary) {
    return salary * TAX_RATE;
}

// Method to display output
public static void displayResults(double gross, double tax, double net) {
    System.out.println("\n----- Tax Summary -----");
    System.out.printf("Gross Salary : $%.2f%n", gross);
    System.out.printf("Tax Rate    : %.2f%%\n", TAX_RATE * 100);
    System.out.printf("Tax Amount   : $%.2f%n", tax);
    System.out.printf("Net Salary   : $%.2f%n", net);
    System.out.println("-----");
}
}

```

Sample Output:

Enter your gross salary: \$60000

```

----- Tax Summary -----
Gross Salary : $60000.00
Tax Rate    : 15.00%
Tax Amount   : $9000.00
Net Salary   : $51000.00
-----

```

Benefits of This Structure:

Feature	Benefit
final constant	Easy to change tax rate globally
Scanner	Interactive user input
Methods	Modular and reusable code
printf()	Neat and formatted output

What are Data Types in Java?

Great! Let's break down **Data Types in Java** in detail with explanations, examples, memory sizes, and when to use which.

◆ What are Data Types in Java?

In Java, **data types** define the type of data a variable can hold. Java is a **statically typed** language, meaning you must declare the type of a variable before using it.

◆ Categories of Data Types

Java has **two main types** of data types:

1. **Primitive Data Types**
 2. **Non-Primitive (Reference) Data Types**
-

✓ 1. Primitive Data Types

These are the **basic building blocks** of data manipulation. There are 8 primitive types:

Type	Size	Default	Example	Description
byte	1 byte	0	byte b = 100;	Whole number from -128 to 127
short	2 bytes	0	short s = 1000;	Whole number from -32,768 to 32,767
int	4 bytes	0	int x = 10000;	Whole number from -2 ³¹ to 2 ³¹ -1
long	8 bytes	0L	long l = 99999L;	Big whole numbers, add L suffix
float	4 bytes	0.0f	float f = 3.14f;	Decimal number, add f suffix
double	8 bytes	0.0	double d = 3.1415;	High-precision decimals
char	2 bytes	'\u0000'	char c = 'A';	A single character
boolean	1 bit	false	boolean b = true;	Only true or false

🔍 Example of All Primitive Data Types:

```
public class PrimitiveTypes {
    public static void main(String[] args) {
        byte b = 100;
        short s = 20000;
        int i = 123456;
        long l = 123456789L;

        float f = 3.14f;
        double d = 3.1415926535;

        char c = 'J';
        boolean isJavaFun = true;

        System.out.println("byte: " + b);
        System.out.println("short: " + s);
        System.out.println("int: " + i);
        System.out.println("long: " + l);
        System.out.println("float: " + f);
    }
}
```

```

        System.out.println("double: " + d);
        System.out.println("char: " + c);
        System.out.println("boolean: " + isJavaFun);
    }
}

```

✔ 2. Non-Primitive (Reference) Data Types

These are not defined by the language but are created by the programmer or from the standard library. Examples:

- **String**
- **Arrays**
- **Classes**
- **Interfaces**
- **Enums**

They **store references (memory addresses)** to objects, not the actual data.

🔍 Example:

```

public class ReferenceTypes {
    public static void main(String[] args) {
        String name = "Java";
        int[] numbers = {1, 2, 3, 4, 5};

        System.out.println("String: " + name);
        System.out.print("Array: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
    }
}

```

🔧 Primitive vs Non-Primitive

Feature	Primitive Types	Non-Primitive Types
Size is fixed?	Yes	No
Stores actual value?	Yes	No (stores reference)
Defined by Java?	Yes (built-in)	No (user or library-defined)
Can be null?	No (except boolean, char have defaults)	Yes
Methods available?	No	Yes (e.g., String.length())

💡 Pro Tips

- Use int for whole numbers unless you specifically need long.
- Use double for decimal values unless you have memory concerns—then use float.
- Use char for single letters, but for most text, use String.
- boolean is useful for conditions (like in if statements).
